

Utilisation de l'interface Runnable

Fabrice.Kordon@lip6.fr



En guise d'introduction...

2

La classe Thread implémente l'interface Runnable

-  Contient la méthode `run()` vide
 - ▶ A surcharger pour définir les traitements (via une sous-classe)

Créer une thread qui hérite d'une classe?

-  Pas d'héritage multiple
 - ▶ Passer au constructeur d'une thread un objet qui implémente Runnable

-  **Étape 1:**
Créer une classe implémentant Runnable

-  **Étape 2:**
Créer un objet à partir de cette classe

-  **Étape 3:**
Créer une thread avec en lui associant cet objet

-  **Étape 4:**
Lancer la thread

C'est l'approche recommandée

Révisons le premier exemple (présentation de la classe Thread)

3

```
class UneTache implements Runnable {
    String monNom;
    UneTache (String n) {
        monNom = n;
    }
    public void run() {
        System.out.println ("Bonjour de " + monNom);
        System.out.println ("Au revoir de " + monNom);
    }
}

public class ExempleRunnable1 {
    public static void main(String []args) {
        Thread t1, t2, t3;
        UneTache c1, c2, c3;
        c1 = new UneTache("t1");
        t1 = new Thread(c1);
        c2 = new UneTache("t2");
        t2 = new Thread(c2);
        c3 = new UneTache("t3");
        t3 = new Thread(c3);
        t1.start();
        t2.start();
        t3.start();
    }
}
```


Révisons le premier exemple (présentation de la classe Thread)

3

```
class UneTache implements Runnable {  
    String monNom;  
    UneTache(String monNom) {  
        this.monNom = monNom;  
    }  
    public void run() {  
        System.out.println("Bonjour de " + monNom);  
        System.out.println("Au revoir de " + monNom);  
    }  
}
```



Remarque

t1 est une Thread
(embarque un UneTache)

```
public class ExempleRunnable1 {  
    public static void main(String []args) {  
        Thread t1, t2, t3;  
        UneTache c1, c2, c3;  
        c1 = new UneTache("t1");  
        t1 = new Thread(c1);  
        c2 = new UneTache("t2");  
        t2 = new Thread(c2);  
        c3 = new UneTache("t3");  
        t3 = new Thread(c3);  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

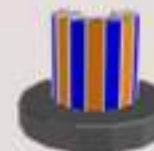


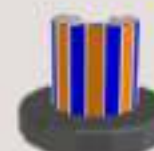
```
$ java ExempleRunnable1  
Bonjour de t1  
Au revoir de t1  
Bonjour de t3  
Au revoir de t3  
Bonjour de t2  
Au revoir de t2  
$ java ExempleRunnable1  
Bonjour de t1  
Au revoir de t1  
Bonjour de t2  
Au revoir de t2  
Bonjour de t3  
Au revoir de t3
```

Même
comportement

Remarque: comment manipuler la thread

4

 **this** inutilisable pour manipuler la thread...

 Pour forcer les commutations ou provoquer des attentes?

 `yield()` et `sleep()` sont des **méthodes de classe**



Second exemple avec commutation forcée

5

```
class UneTache implements Runnable {
    String monNom;
    UneTache (String n) {
        monNom = n;
    }
    public void run() {
        System.out.println ("Bonjour de " + monNom);
        Thread.yield();
        System.out.println ("Au revoir de " + monNom);
    }
}

public class ExempleRunnable2 {
    public static void main(String []args) {
        Thread t1, t2, t3;
        UneTache c1, c2, c3;
        c1 = new UneTache("t1");
        t1 = new Thread(c1);
        c2 = new UneTache("t2");
        t2 = new Thread(c2);
        c3 = new UneTache("t3");
        t3 = new Thread(c3);
        t1.start();
        t2.start();
        t3.start();
    }
}
```


Chronogramme de l'exécution

6

main

```
public class ExempleRunnable2 {  
    public static void main(String []args) {  
        Thread t1, t2, t3;  
        UneTache c1, c2, c3;  
  
        ▶ c1 = new UneTache("t1");  
          t1 = new Thread(c1);  
          c2 = new UneTache("t2");  
          t2 = new Thread(c2);  
          c3 = new UneTache("t3");  
          t3 = new Thread(c3);  
  
          t1.start();  
          t2.start();  
          t3.start();  
    }  
}
```



```
$ java ExempleRunnable2
```

Chronogramme de l'exécution

6



```
public class ExempleRunnable2 {  
    public static void main(String []args) {  
        Thread t1, t2, t3;  
        UneTache c1, c2, c3;  
  
        c1 = new UneTache("t1");  
        t1 = new Thread(c1);  
        c2 = new UneTache("t2");  
        t2 = new Thread(c2);  
        c3 = new UneTache("t3");  
        t3 = new Thread(c3);  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

```
$ java ExempleRunnable2
```


Chronogramme de l'exécution

6



```
public class ExempleRunnable2 {  
    public static void main(String []args) {  
        Thread t1, t2, t3;  
        UneTache c1, c2, c3;  
  
        c1 = new UneTache("t1");  
        t1 = new Thread(c1);  
        c2 = new UneTache("t2");  
        t2 = new Thread(c2);  
        c3 = new UneTache("t3");  
        t3 = new Thread(c3);  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

A terminal window with a dark background and light text. It shows the command to run the Java program: `$ java ExempleRunnable2`. The window has standard macOS window controls (red, yellow, green buttons) in the top left corner.

Chronogramme de l'exécution

6



```
public class ExempleRunnable2 {  
    public static void main(String []args) {  
        Thread t1, t2, t3;  
        UneTache c1, c2, c3;  
  
        c1 = new UneTache("t1");  
        t1 = new Thread(c1);  
        c2 = new UneTache("t2");  
        t2 = new Thread(c2);  
        c3 = new UneTache("t3");  
        t3 = new Thread(c3);  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

A terminal window with a dark background and light text. It shows the command to run the program: `$ java ExempleRunnable2`. The window has three colored buttons (red, yellow, green) in the top left corner.

Chronogramme de l'exécution

6

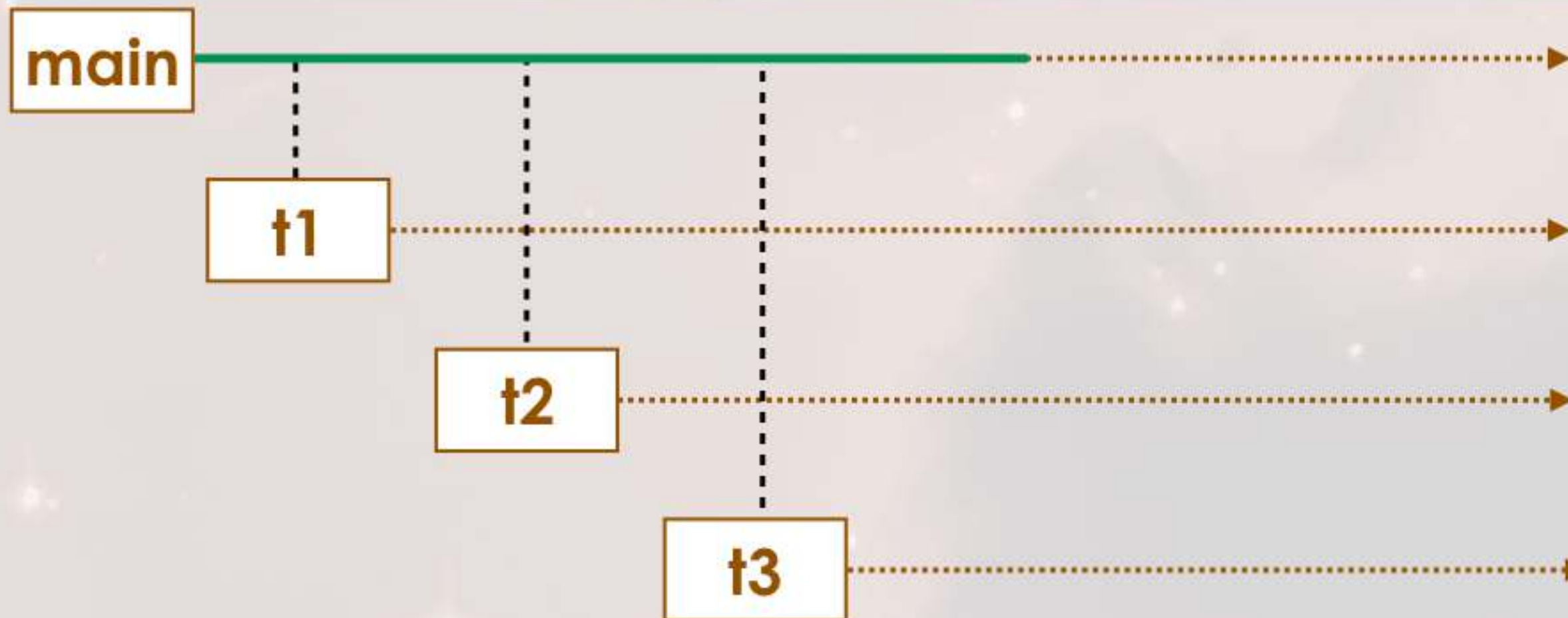


```
public class ExempleRunnable2 {  
    public static void main(String []args) {  
        Thread t1, t2, t3;  
        UneTache c1, c2, c3;  
  
        c1 = new UneTache("t1");  
        t1 = new Thread(c1);  
        c2 = new UneTache("t2");  
        t2 = new Thread(c2);  
        c3 = new UneTache("t3");  
        t3 = new Thread(c3);  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

A terminal window with a dark background and light text. It shows the command to run the program: `$ java ExempleRunnable2`. The window has standard macOS-style window controls (red, yellow, green buttons) in the top-left corner.

Chronogramme de l'exécution

6

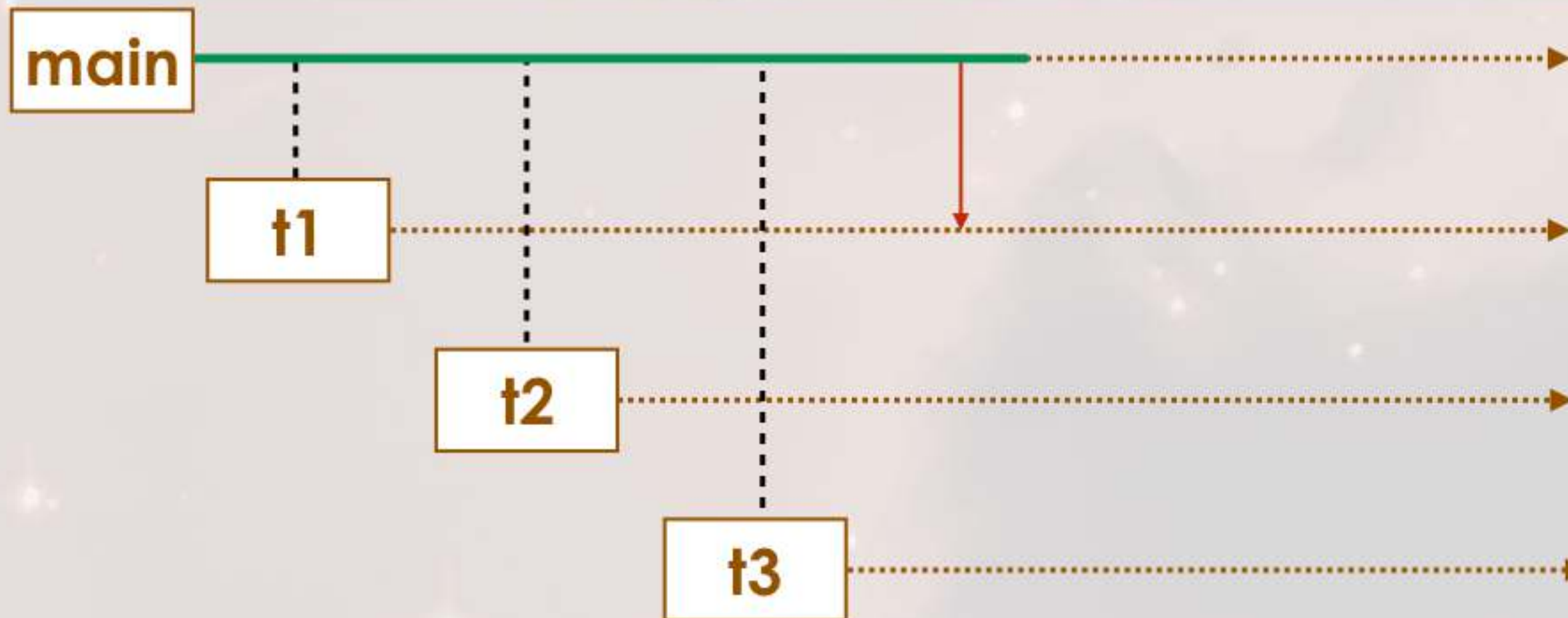


```
public class ExempleRunnable2 {  
    public static void main(String []args) {  
        Thread t1, t2, t3;  
        UneTache c1, c2, c3;  
  
        c1 = new UneTache("t1");  
        t1 = new Thread(c1);  
        c2 = new UneTache("t2");  
        t2 = new Thread(c2);  
        c3 = new UneTache("t3");  
        t3 = new Thread(c3);  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

```
$ java ExempleRunnable2
```


Chronogramme de l'exécution

6

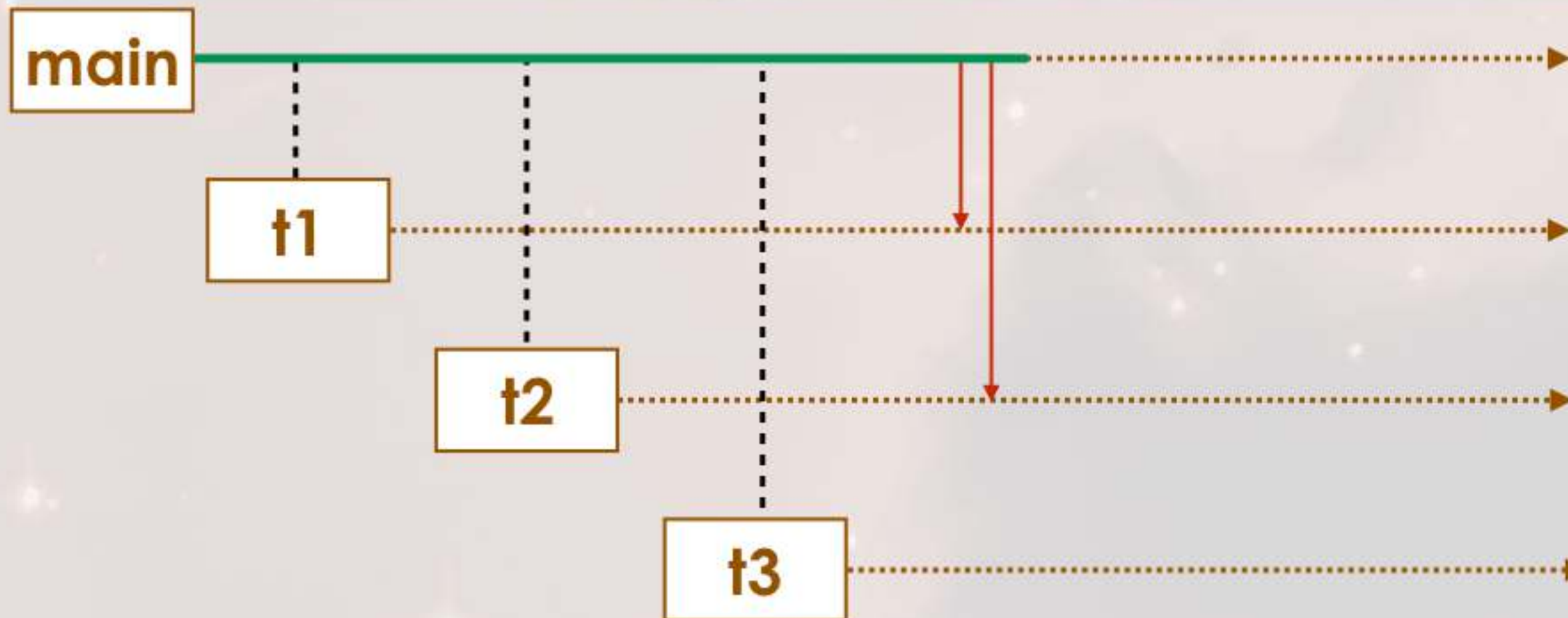


```
public class ExempleRunnable2 {  
    public static void main(String []args) {  
        Thread t1, t2, t3;  
        UneTache c1, c2, c3;  
  
        c1 = new UneTache("t1");  
        t1 = new Thread(c1);  
        c2 = new UneTache("t2");  
        t2 = new Thread(c2);  
        c3 = new UneTache("t3");  
        t3 = new Thread(c3);  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```



Chronogramme de l'exécution

6

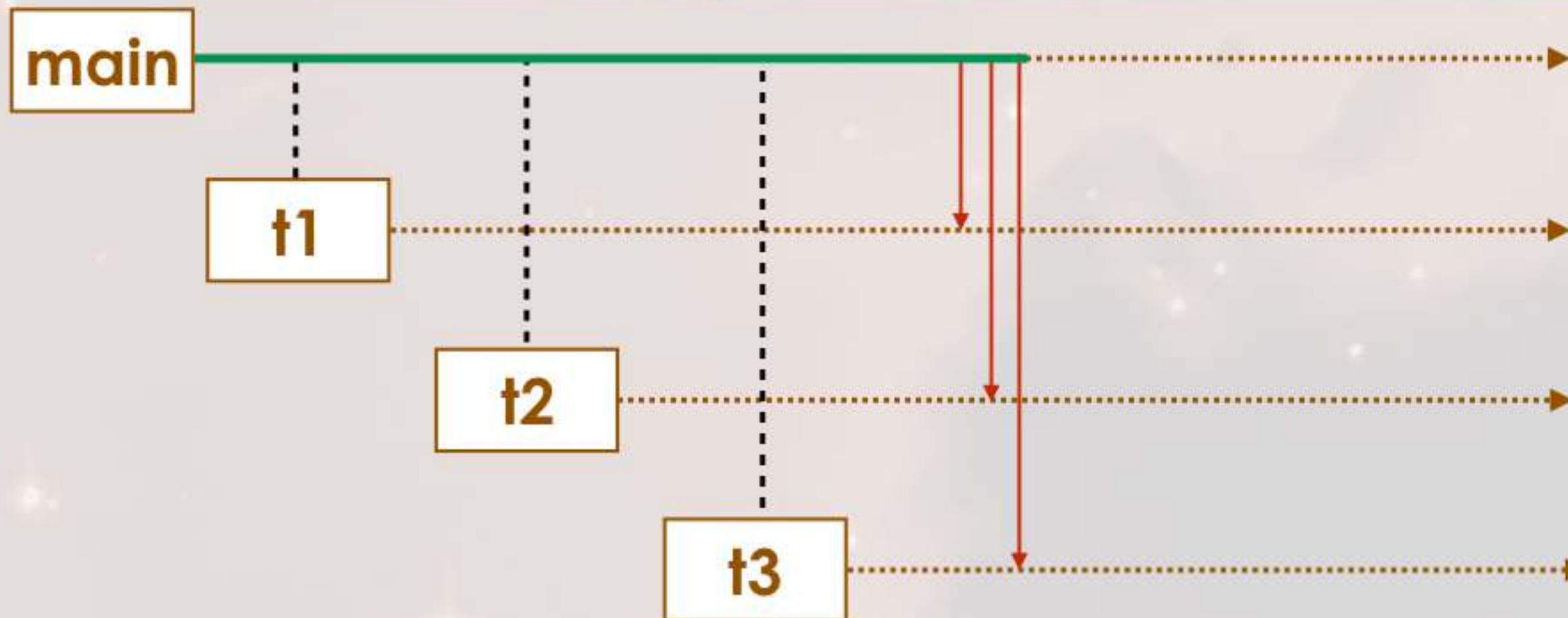


```
public class ExempleRunnable2 {  
    public static void main(String []args) {  
        Thread t1, t2, t3;  
        UneTache c1, c2, c3;  
  
        c1 = new UneTache("t1");  
        t1 = new Thread(c1);  
        c2 = new UneTache("t2");  
        t2 = new Thread(c2);  
        c3 = new UneTache("t3");  
        t3 = new Thread(c3);  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

\$ java ExempleRunnable2

Chronogramme de l'exécution

6

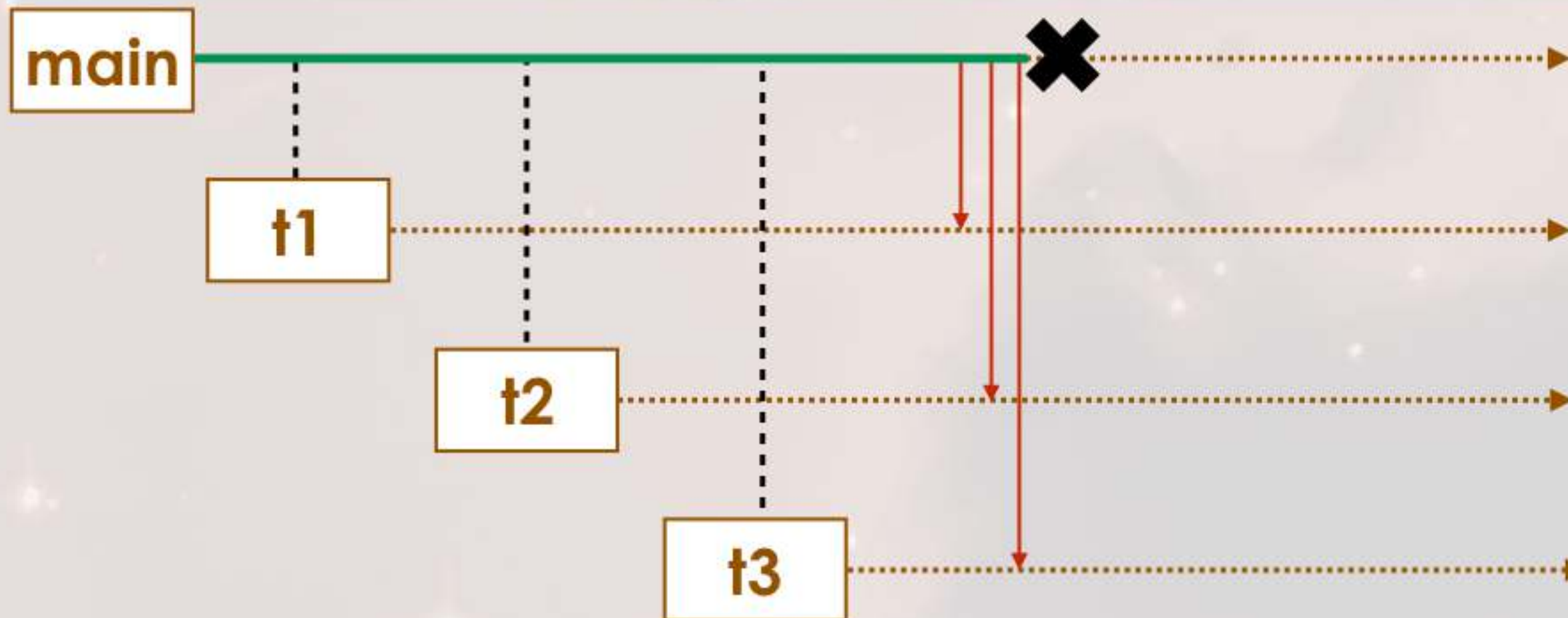


```
public class ExempleRunnable2 {  
    public static void main(String []args) {  
        Thread t1, t2, t3;  
        UneTache c1, c2, c3;  
  
        c1 = new UneTache("t1");  
        t1 = new Thread(c1);  
        c2 = new UneTache("t2");  
        t2 = new Thread(c2);  
        c3 = new UneTache("t3");  
        t3 = new Thread(c3);  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

```
$ java ExempleRunnable2
```


Chronogramme de l'exécution

6

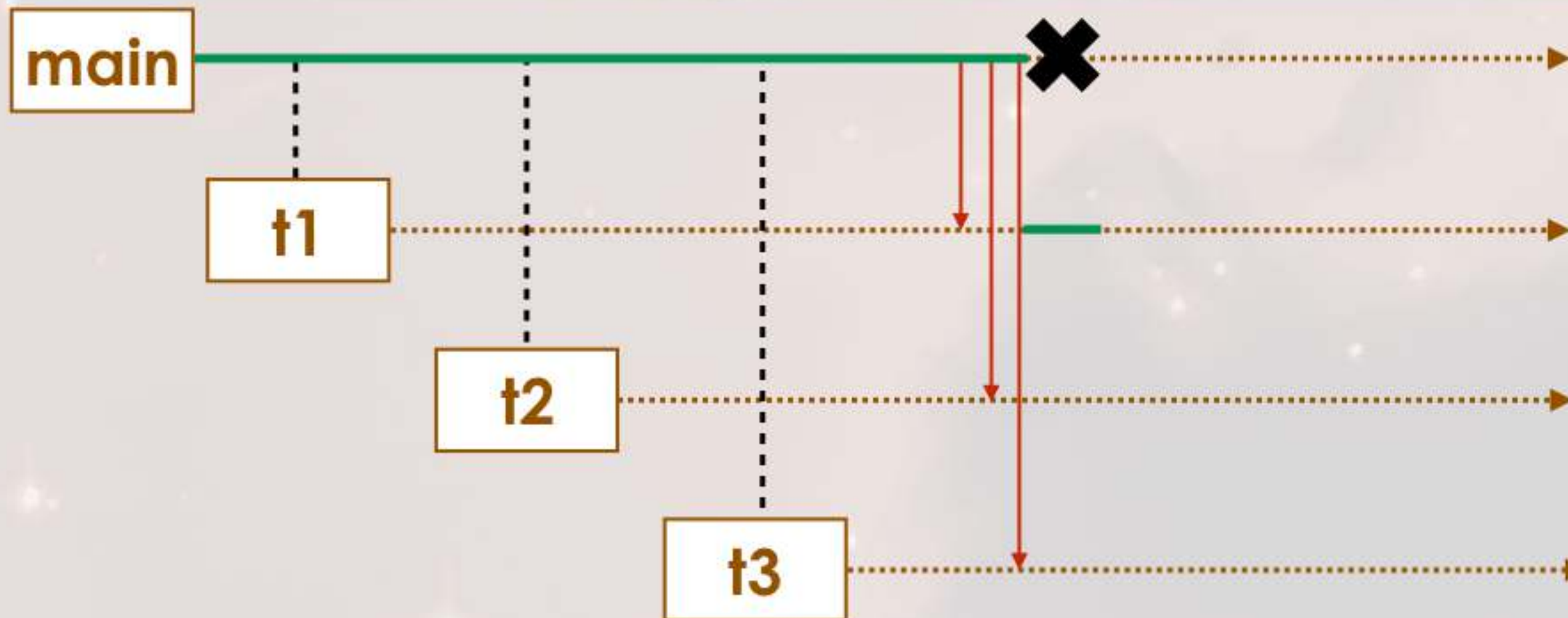


```
class UneTache implements Runnable {  
    String monNom;  
  
    UneTache (String n) {  
        monNom = n;  
    }  
  
    public void run() {  
        System.out.println ("Bonjour de " + monNom);  
        Thread.yield();  
        System.out.println ("Au revoir de " + monNom);  
    }  
}
```

```
$ java ExempleRunnable2
```


Chronogramme de l'exécution

6

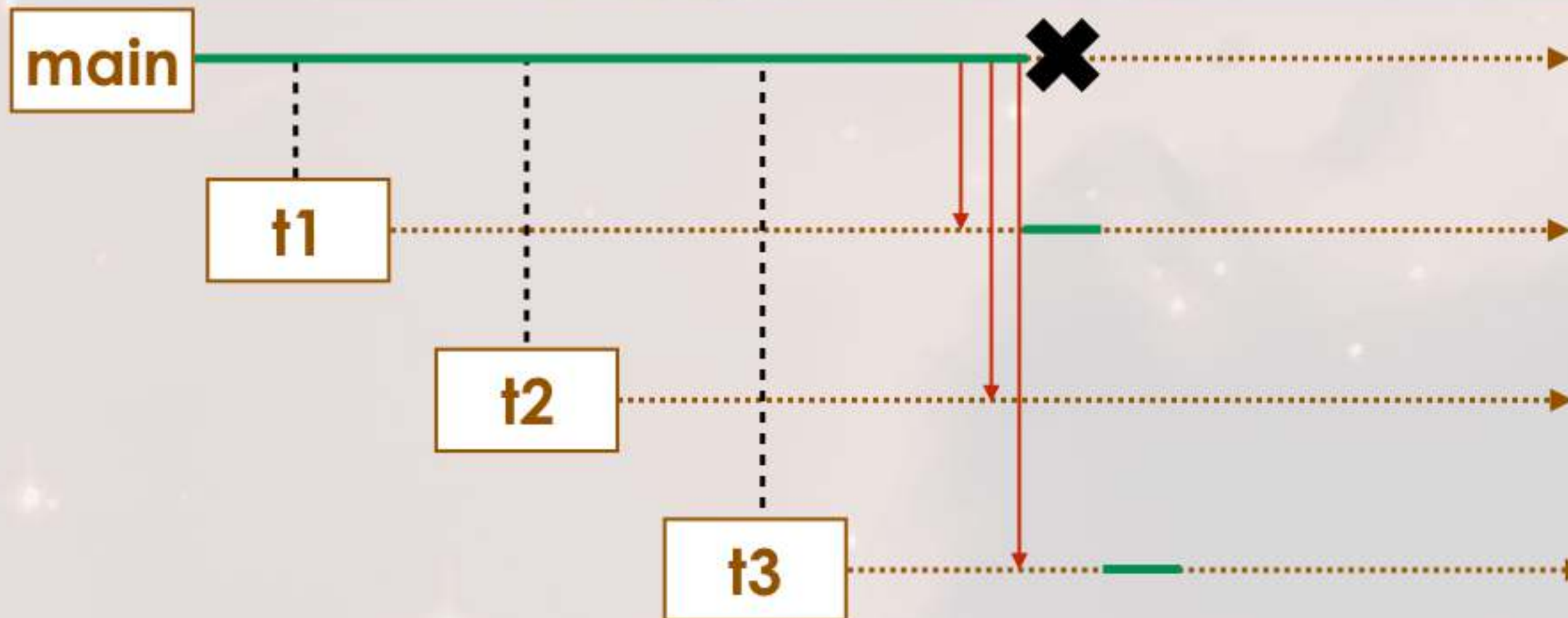


```
class UneTache implements Runnable {  
    String monNom;  
  
    UneTache (String n) {  
        monNom = n;  
    }  
  
    public void run() {  
        System.out.println ("Bonjour de " + monNom);  
        Thread.yield();  
        System.out.println ("Au revoir de " + monNom);  
    }  
}
```

```
$ java ExempleRunnable2  
Bonjour de t1
```


Chronogramme de l'exécution

6

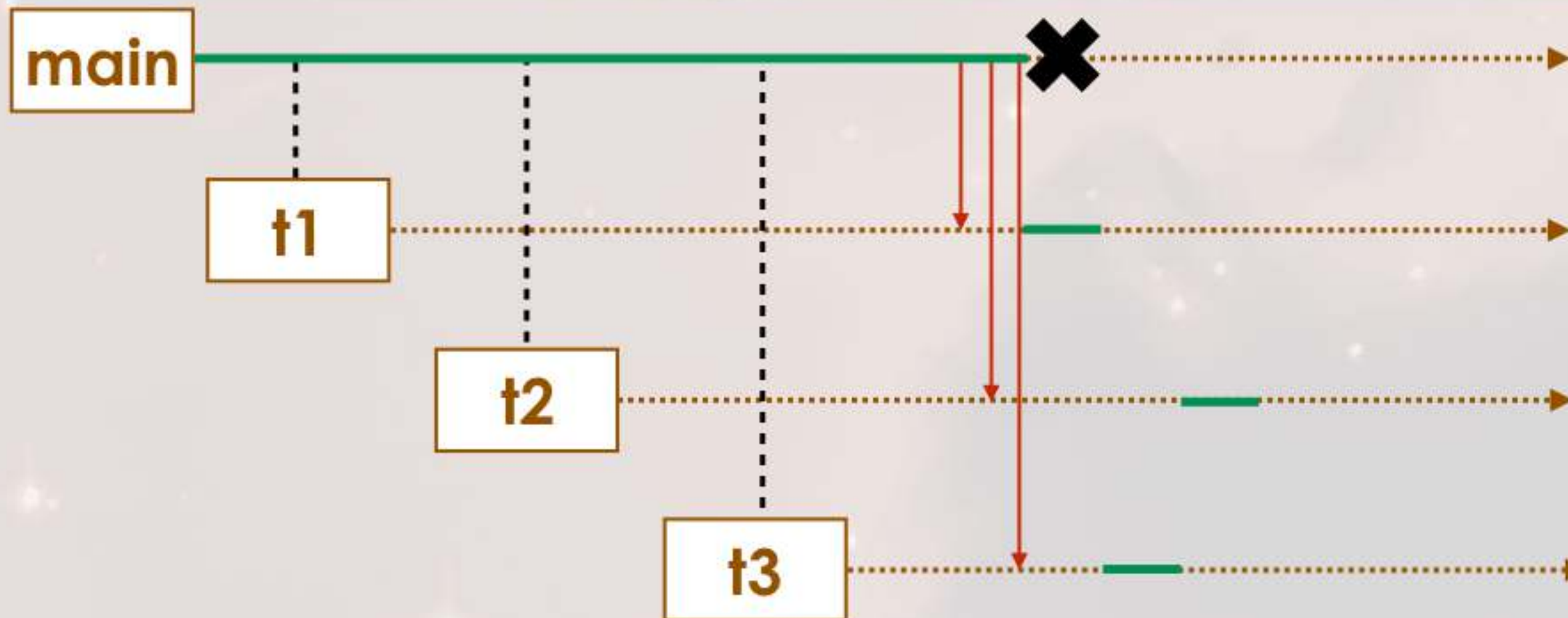


```
class UneTache implements Runnable {  
    String monNom;  
  
    UneTache (String n) {  
        monNom = n;  
    }  
  
    public void run() {  
        System.out.println ("Bonjour de " + monNom);  
        Thread.yield();  
        System.out.println ("Au revoir de " + monNom);  
    }  
}
```

```
$ java ExempleRunnable2  
Bonjour de t1  
Bonjour de t3
```


Chronogramme de l'exécution

6

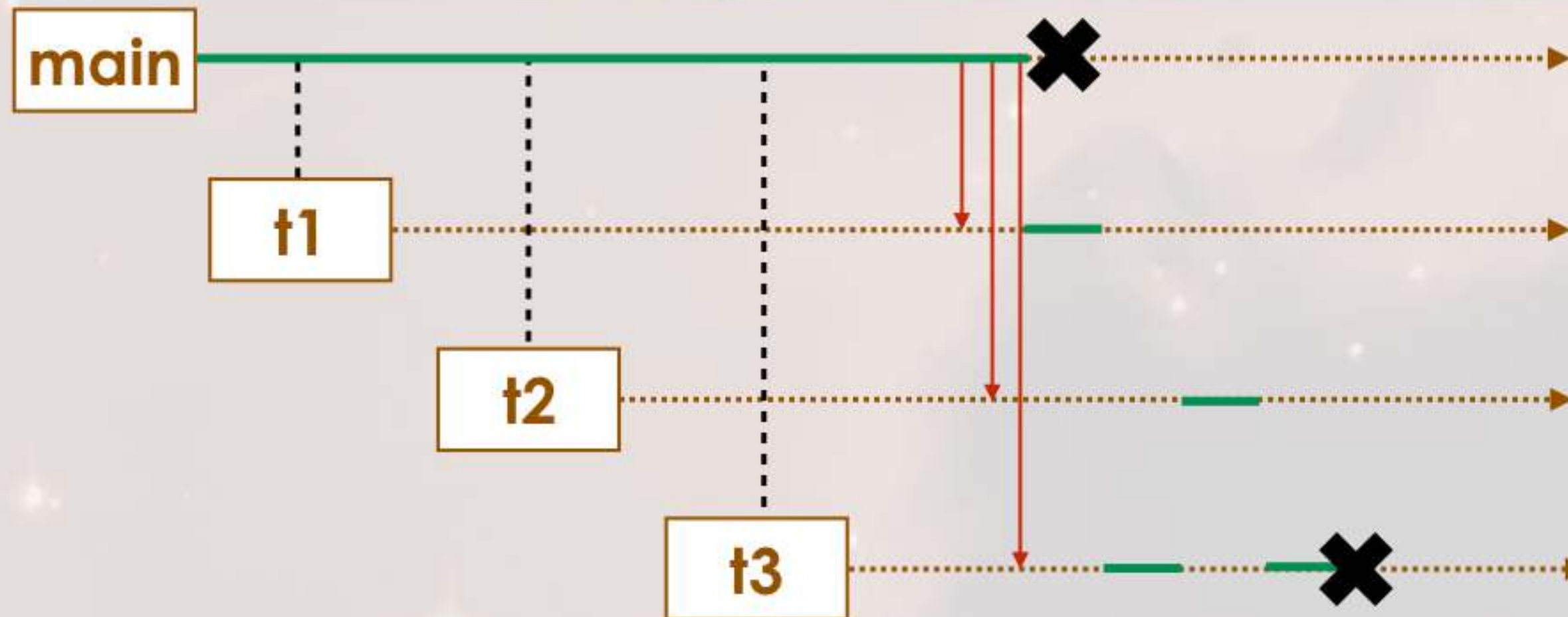


```
class UneTache implements Runnable {  
    String monNom;  
  
    UneTache (String n) {  
        monNom = n;  
    }  
  
    public void run() {  
        System.out.println ("Bonjour de " + monNom);  
        Thread.yield();  
        System.out.println ("Au revoir de " + monNom);  
    }  
}
```

```
$ java ExempleRunnable2  
Bonjour de t1  
Bonjour de t3  
Bonjour de t2
```


Chronogramme de l'exécution

6

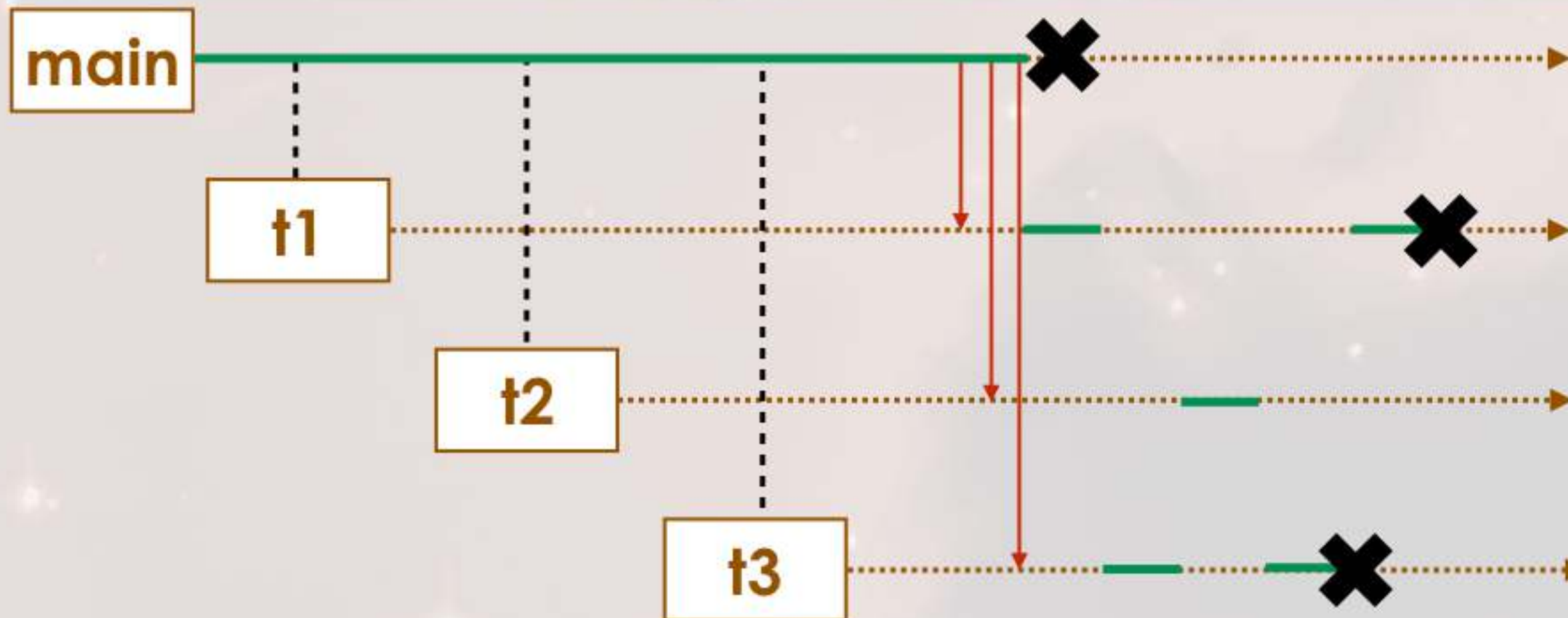


```
class UneTache implements Runnable {  
    String monNom;  
  
    UneTache (String n) {  
        monNom = n;  
    }  
  
    public void run() {  
        System.out.println ("Bonjour de " + monNom);  
        Thread.yield();  
        System.out.println ("Au revoir de " + monNom);  
    }  
}
```

```
$ java ExempleRunnable2  
Bonjour de t1  
Bonjour de t3  
Bonjour de t2  
Au revoir de t3
```


Chronogramme de l'exécution

6

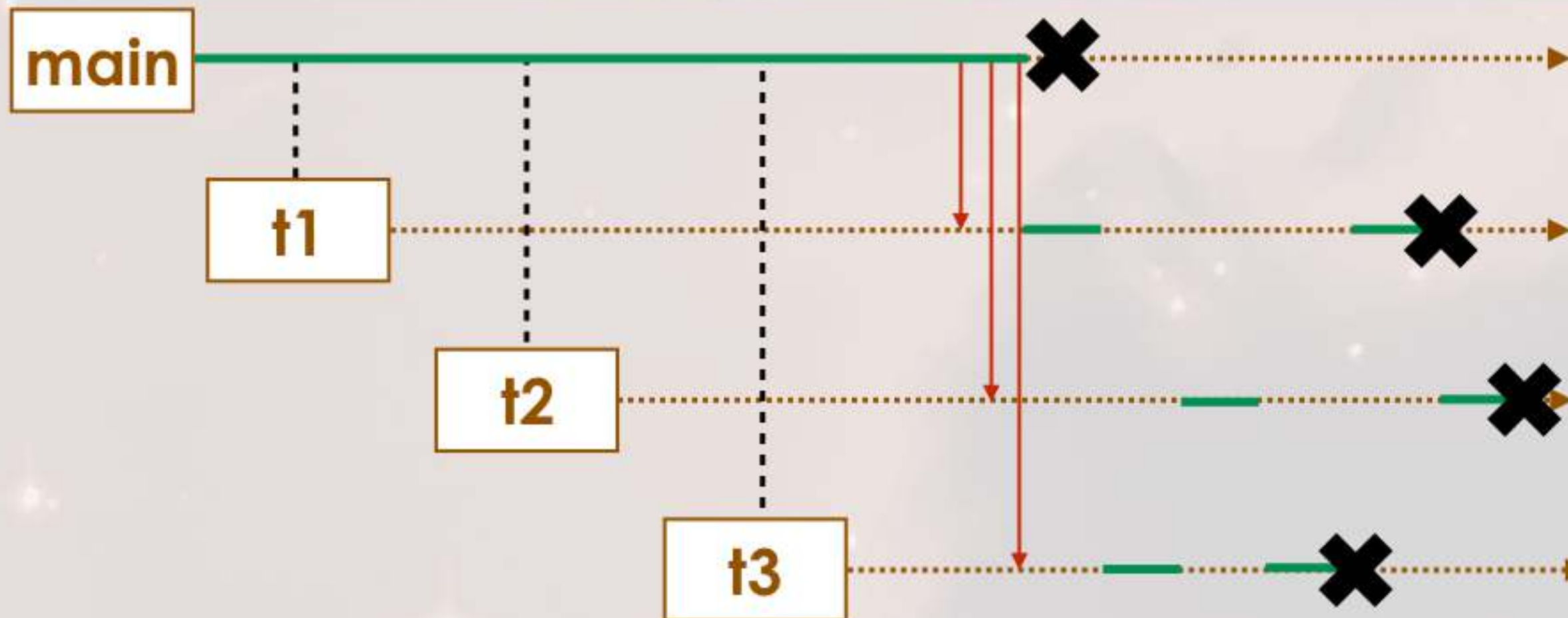


```
class UneTache implements Runnable {  
    String monNom;  
  
    UneTache (String n) {  
        monNom = n;  
    }  
  
    public void run() {  
        System.out.println ("Bonjour de " + monNom);  
        Thread.yield();  
        System.out.println ("Au revoir de " + monNom);  
    }  
}
```

```
$ java ExempleRunnable2  
Bonjour de t1  
Bonjour de t3  
Bonjour de t2  
Au revoir de t3  
Au revoir de t1
```


Chronogramme de l'exécution

6



```
class UneTache implements Runnable {  
    String monNom;  
  
    UneTache (String n) {  
        monNom = n;  
    }  
  
    public void run() {  
        System.out.println ("Bonjour de " + monNom);  
        Thread.yield();  
        System.out.println ("Au revoir de " + monNom);  
    }  
}
```

```
$ java ExempleRunnable2  
Bonjour de t1  
Bonjour de t3  
Bonjour de t2  
Au revoir de t3  
Au revoir de t1  
Au revoir de t2
```


Conclusion sur la manipulation de threads

7

Garder à l'esprit l'extension de la classe Thread

-  Cela est parfois utile
-  Il faut connaître les mécanismes de manipulation de la thread

Mais «l'approche Runnable» est recommandée

-  Directive claire dans les manuels Java

Mais...

-  `run()` est de type void
 - ▶ Pas de gestion des exceptions
-  Interface Callable
 - ▶ Vue plus tard

